

Run Debug Stop Share Save { } Beautify

main.py

```
1 def sequential_search(arr, key):
2     comparisons = 0
3
4     for i in range(len(arr)):
5         comparisons += 1
6         if arr[i] == key:
7             # Position is 1-based index (index + 1)
8             return i + 1, comparisons
9
10    return -1, comparisons
11
12 # Input Data
13 array = [12, 25, 8, 45, 32, 19, 50]
14 search_key = 32
15
16 # Execution
17 position, total_comparisons = sequential_search(array, search_key)
18
19 # Output Setup
20 if position != -1:
21     print(f"Element found at position {position}")
22     print(f"Number of comparisons = {total_comparisons}")
23 else:
24     print("Element not found in the array.")
25     print(f"Number of comparisons = {total_comparisons}")
26
```

input

Element found at position 5

Number of comparisons = 5

...Program finished with exit code 0

Press ENTER to exit console.

```

1 def sequential_search_unsuccessful(arr, key):
2     comparisons = 0
3
4     for i in range(len(arr)):
5         comparisons += 1
6         if arr[i] == key:
7             return i + 1, comparisons
8
9     return -1, comparisons
10
11 # Input Data
12 array = [5, 10, 15, 20, 25, 30, 35]
13 search_key = 18
14
15 # Execution
16 position, total_comparisons = sequential_search_unsuccessful(array, search_key)
17
18 # Output Setup
19 if position != -1:
20     print(f"Element found at position {position}")
21 else:
22     print("Element not found")
23     print(f"Number of comparisons = {total_comparisons}")
24

```

input

Element not found
Number of comparisons = 7

...Program finished with exit code 0
Press ENTER to exit console.

Run Debug Stop Share Save {} Beautify

main.py

```
1 def first_occurrence(arr, key):
2     for i in range(len(arr)):
3         if arr[i] == key:
4             return i + 1
5     return -1
6
7 array = [10, 25, 15, 25, 30, 25, 40]
8 key = 25
9 print(f"First occurrence at position {first_occurrence(array, key)}")
10
```

input

First occurrence at position 2

...Program finished with exit code 0
Press ENTER to exit console.



main.py

```
1 def all_occurrences(arr, key):
2     positions = [i + 1 for i in range(len(arr)) if arr[i] == key]
3     return positions
4
5 array = [7, 12, 7, 25, 18, 7, 30, 7]
6 key = 7
7 pos_list = all_occurrences(array, key)
8 print("Occurrences at positions:")
9 print(", ".join(map(str, pos_list)))
10 print(f"Total occurrences = {len(pos_list)}")
11
```

input

Occurrences at positions:

1, 3, 6, 8

Total occurrences = 4

...Program finished with exit code 0

Press ENTER to exit console.

Run Debug Stop Share Save Beautify

main.py

```
1 def search_metrics(arr, key):
2     comparisons = 0
3     matches = 0
4     mismatches = 0
5     for item in arr:
6         comparisons += 1
7         if item == key:
8             matches += 1
9         else:
10            mismatches += 1
11    return comparisons, matches, mismatches
12
13 array = [3, 6, 9, 12, 15, 18, 21]
14 key = 15
15 comps, matches, mismatches = search_metrics(array, key)
16 print(f"Total comparisons = {comps}")
17 print(f"Total matches = {matches}")
18 print(f"Total mismatches = {mismatches}")
19
```

input

```
Total comparisons = 7
Total matches = 1
Total mismatches = 6
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Run Debug Stop Share Save Beautify

main.py

```
5     if arr[i] == key:
6         return i, comparisons
7     return -1, comparisons
8
9 def sentinel_search(arr, key):
10     n = len(arr)
11     last = arr[n - 1]
12     arr[n - 1] = key # Place sentinel
13     i = 0
14     comparisons = 0
15
16     while True:
17         comparisons += 1
18         if arr[i] == key:
19             break
20         i += 1
21
22     arr[n - 1] = last # Restore
23     if i < n - 1 or arr[n - 1] == key:
24         return i, comparisons
25     return -1, comparisons
26
27 array = [14, 9, 22, 35, 18, 41, 27]
28 key = 18
29
30 ord_idx, ord_comps = ordinary_search(array, key)
31 sent_idx, sent_comps = sentinel_search(array, key)
32
33 print(f"Ordinary Search -> Position found: {ord_idx + 1}, Data Comparisons: {ord_comps}")
34 print(f"Sentinel Search -> Position found: {sent_idx + 1}, Key Comparisons: {sent_comps}")
35 print("Comparison: Sentinel search eliminates the loop index boundary condition evaluation 'i < n'")
36
```

input

```
Ordinary Search -> Position found: 5, Data Comparisons: 5
Sentinel Search -> Position found: 5, Key Comparisons: 5
Comparison: Sentinel search eliminates the loop index boundary condition evaluation 'i < n' at every iteration step.

...Program finished with exit code 0
Press ENTER to exit console.
```




main.py

```
1 def search_reg_no(arr, key):
2     for i, val in enumerate(arr):
3         if val == key:
4             return i + 1
5     return -1
6
7 reg_numbers = [101, 102, 103, 104, 105, 106]
8 key = 104
9 print(f"Register Number found at position {search_reg_no(reg_numbers, key)}")
10
```



input

Register Number found at position 4

...Program finished with exit code 0
Press ENTER to exit console.

Run Debug Stop Share Save Beautify

main.py

```
1 def search_string(arr, key):
2     for i, name in enumerate(arr):
3         if name == key:
4             return i + 1
5     return -1
6
7 names = ["Anu", "Bala", "Charan", "Deepa", "Esha", "Farhan"]
8 search_name = "Deepa"
9 print(f"Name found at position {search_string(names, search_name)}")
10
```

input

Name found at position 4

...Program finished with exit code 0
Press ENTER to exit console.

Run Debug Stop Share Save {} Beautify

main.py

```
1 def search_2d(matrix, key):
2     for r in range(len(matrix)):
3         for c in range(len(matrix[0])):
4             if matrix[r][c] == key:
5                 return r + 1, c + 1
6     return -1, -1
7
8 matrix = [
9     [9, 11, 24]
10 ]
11 key = 24
12 row, col = search_2d(matrix, key)
13 print(f"Element found at Row {row} Column {col}")
14
```

input

input

Element found at Row 1 Column 3

...Program finished with exit code 0
Press ENTER to exit console.

main.py

```
1 def analyze_sequential(arr, keys):
2     for key in keys:
3         print(f"\n--- Searching for Key: {key} ---")
4         comparisons = 0
5         found = False
6         for i in range(len(arr)):
7             comparisons += 1
8             print(f"Comparison {comparisons}: Checking index {i} (Value: {arr[i]}) == {key}")
9             if arr[i] == key:
10                 print(f"-> Element found at position {i+1} with {comparisons} comparisons.")
11                 found = True
12                 break
13         if not found:
14             print(f"-> Element not found. Total comparisons: {comparisons}")
15
16 array = [45, 23, 67, 12, 89, 34, 56, 78, 90, 11, 29, 73, 18, 64, 37]
17 search_keys = [73, 18, 100]
18 analyze_sequential(array, search_keys)
19
```

input

```
Comparison 12: Checking index 11 (Value: 73) == 100
Comparison 13: Checking index 12 (Value: 18) == 100
Comparison 14: Checking index 13 (Value: 64) == 100
Comparison 15: Checking index 14 (Value: 37) == 100
-> Element not found. Total comparisons: 15
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```



main.py

```
1 def brute_force_string_match(text, pattern):
2     n, m = len(text), len(pattern)
3     total_comparisons = 0
4     occurrences = []
5
6     for i in range(n - m + 1):
7         match = True
8         for j in range(m):
9             total_comparisons += 1
10            if text[i + j] != pattern[j]:
11                match = False
12                break
13        if match:
14            occurrences.append(i)
15    return occurrences, total_comparisons
16
17 text = "AABAACAADAABAABA"
18 pattern = "AABA"
19 pos, comps = brute_force_string_match(text, pattern)
20 print(f"Position(s) where the pattern occurs: {pos}")
21 print(f"Total number of comparisons: {comps}")
22
```

input

Position(s) where the pattern occurs: [0, 9, 12]
Total number of comparisons: 30

...Program finished with exit code 0
Press ENTER to exit console.



main.py

```
1 # 1. Define the missing function first
2 def brute_force_string_match(text, pattern):
3     n, m = len(text), len(pattern)
4     total_comparisons = 0
5     occurrences = []
6
7     for i in range(n - m + 1):
8         match = True
9         for j in range(m):
10             total_comparisons += 1
11             if text[i + j] != pattern[j]:
12                 match = False
13                 break
14         if match:
15             occurrences.append(i)
16     return occurrences, total_comparisons
17
18 # 2. Reusing fundamental Loop patterns
19 text = "BANANABANANA"
20 pattern = "ANA"
21
22 # 3. Call the function and unpack results
23 pos, _ = brute_force_string_match(text, pattern)
24 print(f"Occurrences at positions {'', '.join(map(str, pos))}")
25
```

input

Occurrences at positions 1, 3, 7, 9

...Program finished with exit code 0
Press ENTER to exit console.

Run Debug Stop Share Save Beautify

main.py

```
1 def alignment_table(text, pattern):
2     n, m = len(text), len(pattern)
3     print(f"{'Shift':<7}{'Comparisons':<15}{'Result'}")
4     for i in range(n - m + 1):
5         comps = 0
6         match = True
7         for j in range(m):
8             comps += 1
9             if text[i + j] != pattern[j]:
10                 match = False
11                 break
12         res = "Match" if match else "Mismatch"
13         print(f"{'i':<7}{'comps':<15}{res}")
14
15 text = "MISSISSIPPI"
16 pattern = "ISSI"
17 alignment_table(text, pattern)
18
```

input

3	1	Mismatch
4	4	Match
5	1	Mismatch
6	1	Mismatch
7	2	Mismatch

...Program finished with exit code 0
Press ENTER to exit console.

Run Debug Stop Share Save {} Beautify

main.py

```
1 def metric_string_match(text, pattern):
2     n, m = len(text), len(pattern)
3     total_comps = 0
4     matches = 0
5     mismatches = 0
6
7     for i in range(n - m + 1):
8         for j in range(m):
9             total_comps += 1
10            if text[i + j] == pattern[j]:
11                matches += 1
12            else:
13                mismatches += 1
14                break
15        return total_comps, matches, mismatches
16
17 text = "ABABABABAB"
18 pattern = "ABAB"
19 tc, m, mm = metric_string_match(text, pattern)
20 print(f"Total character comparisons = {tc}")
21 print(f"Total matches = {m}")
22 print(f"Total mismatches = {mm}")
23
```

input

```
Total character comparisons = 19
Total matches = 16
Total mismatches = 3

...Program finished with exit code 0
Press ENTER to exit console.
```


main.py

```
1 # 1. Define the missing function first
2 def brute_force_string_match(text, pattern):
3     n, m = len(text), len(pattern)
4     total_comparisons = 0
5     occurrences = []
6
7     for i in range(n - m + 1):
8         match = True
9         for j in range(m):
10             total_comparisons += 1
11             if text[i + j] != pattern[j]:
12                 match = False
13                 break
14         if match:
15             occurrences.append(i + 1) # 1-based indexing
16     return occurrences, total_comparisons
17
18 # 2. Evaluates shift performance properties
19 text = "AAAAAAAAAB"
20 pattern = "AAAAB"
21
22 # 3. Execution
23 pos, comps = brute_force_string_match(text, pattern)
24
25 # 4. Output Display
26 print(f"Comparisons performed: {comps}")
27 print("Case evaluation: Worst Case scenario because for every shift position, all previous match val
28
```

Language Python 3

close ad [x]

Adobe

Creative Cloud Pro

Build career-ready skills with pro apps.

Now ₹399/mo for Students

Learn more

Client wants more options? Done.

Pay via UPI

Buy now

Generative Recreator

Visualize

Shubho Naruka, Illustrator

Adobe Creative Cloud Pro

input

Comparisons performed: 30
Case evaluation: Worst Case scenario because for every shift position, all previous match validations succeed only to fail on the final character window boundaries.

...Program finished with exit code 0
Press ENTER to exit console.

Run Debug Stop Share Save Beautify

main.py

```
1 def first_occurrence_string(text, pattern):
2     n, m = len(text), len(pattern)
3     comps = 0
4     for i in range(n - m + 1):
5         match = True
6         for j in range(m):
7             comps += 1
8             if text[i + j] != pattern[j]:
9                 match = False
10                break
11        if match:
12            return i, comps
13    return -1, comps
14
15 text = "COMPUTERSCIENCE"
16 pattern = "SCI"
17 pos, comps = first_occurrence_string(text, pattern)
18 print(f"First occurrence position: {pos}")
19 print(f"Number of comparisons: {comps}")
20
```

input

First occurrence position: 8
Number of comparisons: 11

...Program finished with exit code 0
Press ENTER to exit console.



main.py

```
1 # 1. Define the core brute-force matching engine
2 def brute_force_string_match(text, pattern):
3     n, m = len(text), len(pattern)
4     total_comparisons = 0
5     occurrences = []
6
7     for i in range(n - m + 1):
8         match = True
9         for j in range(m):
10             total_comparisons += 1
11             if text[i + j] != pattern[j]:
12                 match = False
13                 break
14         if match:
15             occurrences.append(i + 1) # 1-based indexing for position
16     return occurrences, total_comparisons
17
18 # 2. Define your case-insensitive wrapper
19 def case_insensitive_match(text, pattern):
20     t_lower = text.lower()
21     p_lower = pattern.lower()
22     pos = brute_force_string_match(t_lower, p_lower)
23     return pos
24
25 # 3. Input Data
26 text = "DataStructuresAndAlgorithms"
27 pattern = "ALGORITHMS"
28
29 # 4. Execution & Output Display
30 positions, _ = case_insensitive_match(text, pattern)
31 print(f"Pattern found position: {positions}")
32
```

input

Pattern found position: [18]

...Program finished with exit code 0

Press ENTER to exit console.



main.py

```
1 # 1. Define the missing matching function first
2 def brute_force_string_match(text, pattern):
3     n, m = len(text), len(pattern)
4     total_comparisons = 0
5     occurrences = []
6
7     for i in range(n - m + 1):
8         match = True
9         for j in range(m):
10             total_comparisons += 1
11             if text[i + j] != pattern[j]:
12                 match = False
13                 break
14         if match:
15             occurrences.append(i + 1)
16     return occurrences, total_comparisons
17
18 # 2. Input Data
19 text = "PROGRAMMINGLAB"
20 p1 = "LAB"
21 p2 = "TEST"
22
23 # 3. Execution
24 _, comps_success = brute_force_string_match(text, p1)
25 _, comps_fail = brute_force_string_match(text, p2)
26
27 # 4. Output Setup
28 print(f"Successful Search ('{p1}') comparisons: {comps_success}")
29 print(f"Unsuccessful Search ('{p2}') comparisons: {comps_fail}")
30
```

input

```
Successful Search ('LAB') comparisons: 14
Unsuccessful Search ('TEST') comparisons: 11
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```




```
main.py
1 def display_alignments(text, pattern):
2     n, m = len(text), len(pattern)
3     occurrences = []
4     print(f"Text: {text}\n")
5
6     for i in range(n - m + 1):
7         match = True
8         for j in range(m):
9             if text[i + j] != pattern[j]:
10                match = False
11                break
12         res = "MATCH FOUND" if match else "MISMATCH"
13         if match: occurrences.append(i)
14
15         # Format string visual output matrix projection alignment
16         alignment_str = " " * i + pattern
17         print(f"Alignment {i+1:<2}: {alignment_str:<15} -> {res}")
18
19     print(f"\nPattern occurrence positions: {occurrences}")
20
21 text = "ABCDABCABCD"
22 pattern = "ABCD"
23 display_alignments(text, pattern)
24
```

input

```
Alignment 6 :      ABCDA      -> MISMATCH
Alignment 7 :      ABCDA      -> MISMATCH
Alignment 8 :      ABCDA      -> MATCH FOUND
```

```
Pattern occurrence positions: [0, 7]
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Run Debug Stop Share Save Beautify

main.py

```
1 text = "TTATAGATCTCGTATTCTTTATAGATCTCCTATTCTT"
2 pattern = "TATCTT"
3
4 # 1, 2, 3: Occurrences, counts and alignment shifts tracking
5 def comprehensive_analysis(text, pattern):
6     n, m = len(text), len(pattern)
7     occurrences = []
8     total_comps = 0
9
10    print("--- Shifting Process Output Window ---")
11    for i in range(n - m + 1):
12        comps = 0
13        match = True
14        for j in range(m):
15            comps += 1
16            total_comps += 1
17            if text[i + j] != pattern[j]:
18                match = False
19                break
20        if match:
21            occurrences.append(i)
22            print(f"Shift {i:02d}: text[{i}:{i+m}] vs pattern -> Checked {comps} chars -> {'Match' if m}")
23
24    print("\n--- Summary Results ---")
25    print(f"All occurrences found at indices: {occurrences}")
26    print(f"Total character comparisons made: {total_comps}")
27
28    comprehensive_analysis(text, pattern)
29
```

input

Shift 31: text[31:37] vs pattern -> Checked 1 chars -> Mismatch

--- Summary Results ---

All occurrences found at indices: []

Total character comparisons made: 57

...Program finished with exit code 0

Press ENTER to exit console.